核心语言的操作语义

这里的核心语言是编程语言中的 core language。AIC 与 AIV 是硬件核。本文给出一个 AI Core 上的语法、良构条件和小步语义，不改写另两份笔记中的语法与 trace 公理，也不声称与那些公理等价。

1 范围

一项程序至多包含同一个 AI Core 里的两条指令流：AIC 一条，AIV 一条。纯 AIV 或纯 AIC 的 kernel 只写其中一条。多个 AI Core 组成的 block 网格，以及卡与卡之间的 rank，不在这项里。`get_block_idx` 是求值时已经给定的整数。

并发有两层。一个核内部的 `set_flag`、`wait_flag` 和随后的 `load` 写在同一条指令序列里，不分成并行分支。`PIPE_MTE2`、`PIPE_V`、`PIPE_MTE3` 等 pipe 的并行表现为各 pipe 的队列：指令进入队列后，任一已就绪的完成都可以发生。核与核之间才使用 $\|$。两支共享 Global Memory，各有自己的环境、核内 flag 表和 pipe 队列。UB 只出现在 AIV 的足迹里，L1 与 L0C 只出现在 AIC 的足迹里。

`mmadL1` 的内部事件槽不展开。AIC 写到 Global Memory 的一次 `fixpipe` 是一条 issue，用来表达 attention 中 AIC 与 AIV 的数据交接。

2 语法

管道名分成数据管道与两个同步名。

$$\begin{aligned}\mathsf{Pipe}_{\mathsf{AIV}} &= \{\mathsf{MTE2}, \mathsf{V}, \mathsf{MTE3}\},\\ \mathsf{Pipe}_{\mathsf{AIC}} &= \{\mathsf{MTE1}, \mathsf{M}, \mathsf{FIX}\},\\ \mathsf{SyncName} &= \{\mathsf{S}, \mathsf{ALL}\}.\end{aligned}$$

d 取值 AIV 或 AIC。p, q 取自相应的数据管道或同步名，由各条规则限定。

值的类别是 $\tau ::= \mathsf{int} \mid \mathsf{float} \mid \mathsf{bool} \mid \mathsf{event} \mid \mathsf{view}$。常量 c 按字面带有其中一种类别，x 是变量，v 是常量或变量，$\bar{v}$ 是有限序列。end 只作为 for 循环体的迭代结束标记。

$$\begin{aligned}C ::= \;& \mathsf{return}(\bar{v}) \mid \mathsf{end}\\ \mid\; & \mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C\\ \mid\; & \mathsf{issue}[p](f, \bar{v});\ C\\ \mid\; & \mathsf{setflag}[p, q](v);\ C \mid \mathsf{waitflag}[p, q](v);\ C\\ \mid\; & \mathsf{pipebarrier}[p];\ C\\ \mid\; & \mathsf{syncblockset}[p, q](v);\ C \mid \mathsf{syncblockwait}[p, q](v);\ C\\ \mid\; & \mathsf{if}\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\}\\ \mid\; & \mathsf{for}\ x = v_1\ \mathsf{to}\ v_2\ \mathsf{step}\ v_3\ \mathsf{do}\ \{C_1\};\ C_2,\\ P ::= \;& \mathsf{aiv}\{C\} \mid \mathsf{aic}\{C\} \mid \mathsf{aic}\{C_1\} \parallel \mathsf{aiv}\{C_2\}.\end{aligned}$$

issue 不绑定数据结果。地址不是语法类别。存储单元 $\mathsf{Loc} = \mathsf{Space} \times \mathbb{Z}$ 只出现在语义里，其中 $\mathsf{Space} = \{\mathsf{GM}, \mathsf{UB}, \mathsf{L1}, \mathsf{L0C}\}$。

3 良构

Γ 把有限个变量映到类别。标量算子由 $\Delta(g) = (\bar{\tau}, \tau)$ 给出参数类别和结果类别。数据算子由 $\Sigma(f, p) = \bar{\tau}$ 给出：在管道 p 上发射 f 时参数应有的类别。两支 $\parallel$ 各自从空环境推导，环境不共享。

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \qquad \frac{c \text{ 的类别为 } \tau}{\Gamma \vdash c : \tau}$$

$$\frac{\Delta(g) = (\bar{\tau}, \tau) \quad \Gamma \vdash \bar{v} : \bar{\tau} \quad x \notin \mathrm{dom}(\Gamma) \quad \Gamma, x : \tau \vdash_d C}{\Gamma \vdash_d \mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C}$$

$$\frac{p \in \mathsf{Pipe}_d \quad \Sigma(f, p) = \bar{\tau} \quad \Gamma \vdash \bar{v} : \bar{\tau} \quad \Gamma \vdash_d C}{\Gamma \vdash_d \mathsf{issue}[p](f, \bar{v});\ C}$$

核内同步要求两端都是本核的数据管道，且 $p \neq q$。块同步的置位命令要求置位管道 p 属于本核；等待管道可以是 S。等待命令出现在执行等待的那一侧，因此 p 可以是另一核的数据管道。

$$\frac{p, q \in \mathsf{Pipe}_d \quad p \neq q \quad \Gamma \vdash v : \mathsf{event} \quad \Gamma \vdash_d C}{\Gamma \vdash_d \mathsf{setflag}[p, q](v);\ C}$$

$$\frac{p \in \mathsf{Pipe}_d \quad q \in \mathsf{Pipe}_d \cup \{\mathsf{S}\} \quad p \neq q \quad \Gamma \vdash v : \mathsf{event} \quad \Gamma \vdash_d C}{\Gamma \vdash_d \mathsf{syncblockset}[p, q](v);\ C}$$

waitflag 与 setflag 的前提相同。syncblockwait$[p, q](v)$ 要求 $q \in \mathsf{Pipe}_d \cup \{\mathsf{S}\}$、$p \neq q$，并且 v 为 event；p 不必属于 d。pipebarrier$[p]$ 要求 $p \in \mathsf{Pipe}_d \cup \{\mathsf{ALL}\}$。

if 的条件为 bool，两支在同一个 Γ 下良构。for 的三个界为 int，循环变量是新名字，循环体与循环后的命令都良构。循环体不含 for。循环体里每条不以 return 结束的路径以 end 结束。end 与 return$(\bar{v})$ 在任意 Γ 下良构。

$$\frac{\varnothing \vdash_{\mathsf{AIC}} C_1 \quad \varnothing \vdash_{\mathsf{AIV}} C_2}{\vdash \mathsf{aic}\{C_1\} \parallel \mathsf{aiv}\{C_2\}}$$

单独一支 aiv$\{C\}$ 或 aic$\{C\}$ 要求相应的空环境推导。良构不要求 flag 已经置位，不要求存储已初始化，也不要求足迹互不相交。缺少同步的程序仍是合法项。

4 状态与参数

一个核的队列 Q_d 把该核的每条数据管道映到有限序列。队列项是

$$\begin{aligned} \iota ::= \ & \mathsf{op}(f, R, W) \mid \mathsf{set}(q, \varepsilon) \mid \mathsf{wait}(p, \varepsilon) \\ & \mid \mathsf{bar} \mid \mathsf{bset}(q, \varepsilon) \mid \mathsf{bwait}(p, \varepsilon). \end{aligned}$$

R, W 是有限的 Loc 集合。存储 μ 把 Loc 部分映到 Uninit 或值。每个核有自己的核内 flag 表 φ_d，两个核共用块同步表 β。二者都把数据管道、等待管道与事件标识映到 $\{0, 1\}$，初值为 0。

配置含共享的 μ 与 β，以及每一侧的 φ_d、Q_d、环境 ρ_d 和控制 C_d。单独一支程序只有一侧。实例化提供三个部分。eval_g 在与 $\Delta(g)$ 相符的值上得到结果。$\mathsf{foot}(f, \bar{\nu}) = (R, W)$ 给出足迹，并且满足 $R \cup W \subseteq \mathsf{Loc}_d$：AIV 只使用 GM 与 UB，AIC 只使用 GM、L1 与 L0C。关系 $F(f, \mu|_R, \mu')$ 成立时，μ' 与 μ 只在 W 上不同。F 可以有多个结果。

队列中位于等待项或屏障项之后的项尚未打开。op 在自身已经打开时可以完成，不要求位于队首。set、bset 与 bar 只有在前面的项都已离开队列时才能完成。

5 小步规则

一步只前进一个核，可以改写共享的 μ 与 β。下面省去未改变的分量。控制规则使用当前环境 ρ。

标量求值后继续。if 按布尔值进入一支。return 没有控制后继；该核的队列仍可完成。

$$\frac{\bar{\nu} = \rho(\bar{v}) \quad \mathsf{eval}_g(\bar{\nu}) = \nu}{\mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C \longrightarrow \rho[x \mapsto \nu],\ C}$$

发射把一项加到目标管道的队尾，控制立即继续。足迹在发射时确定。foot 或 eval 没有结果时，相应的控制规则不启用。

$$\frac{\mathsf{foot}(f, \rho(\bar{v})) = (R, W)}{Q(p),\ \mathsf{issue}[p](f, \bar{v});\ C \longrightarrow Q(p)\cdot\mathsf{op}(f, R, W),\ C}$$

$\mathsf{setflag}[p, q](\varepsilon)$ 把 $\mathsf{set}(q, \varepsilon)$ 放入 p。$\mathsf{waitflag}[p, q](\varepsilon)$ 把 $\mathsf{wait}(p, \varepsilon)$ 放入 q，不挡住发射流。$\mathsf{pipebarrier}[p]$ 在 p 放入 bar；$p = \mathsf{ALL}$ 时在本核每条数据管道放入 bar。$\mathsf{syncblockset}[p, q](\varepsilon)$ 把 $\mathsf{bset}(q, \varepsilon)$ 放入 p。等待管道不是 S 的 syncblockwait 把 bwait 放入本核的等待管道。

等待管道为 S 时，块等待留在控制位置，该核不能再发射。令牌为 1 时消耗令牌并继续。另一核以及本核已经入队的完成不受这条约制。

$$\frac{\beta(p, \mathsf{S}, \varepsilon) = 1}{\beta,\ \mathsf{syncblockwait}[p, \mathsf{S}](\varepsilon);\ C \longrightarrow \beta[(p, \mathsf{S}, \varepsilon) \mapsto 0],\ C}$$

数据项完成时读取当时的 $\mu|_R$。其中可以有 Uninit，也可以有更早写入的值。规则不检查其他在途指令。

$$\frac{\mathsf{op}(f, R, W)\ \text{已打开} \quad F(f, \mu|_R, \mu')}{\mu,\ Q \longrightarrow \mu',\ Q \setminus \mathsf{op}(f, R, W)}$$

$\mathsf{set}(q, \varepsilon)$ 位于 p 的队首且 $\varphi_d(p, q, \varepsilon) = 0$ 时，该项离开队列，令牌改为 1。φ_d 已为 1 时这条规则不启用。$\mathsf{wait}(p, \varepsilon)$ 在 q 上已经打开且令牌为 1 时离开队列，令牌改回 0；令牌为 0 时不启用。打开条件使排在等待后面的同管道项不能先完成。bar 只在位于队首时离开。bset 与 set 相同，但读写的是 β。bwait 与 wait 相同，同样读写 β。核内表 φ_d 与块同步表 β 不混用。

循环按整数范围展开。$k_3 > 0$ 且 $k_1 < k_2$ 时，把循环体中的 end 换成下一次迭代，并绑定本轮的 x。范围为空时进入循环后的命令。$C \triangleright D$ 表示这个替换；return 保持不变，if 的两支分别替换。

$$\frac{\rho(v_i) = k_i \quad k_3 > 0 \quad k_1 < k_2}{\mathsf{for}\ x = v_1\ \mathsf{to}\ v_2\ \mathsf{step}\ v_3\ \mathsf{do}\ \{C\};\ C' \longrightarrow \mathsf{let}\ x = k_1\ \mathsf{in}\ C \triangleright R}$$

其中 R 是下界改为 $k_1 + k_3$ 的同一个 for。负步长与范围为空的规则方向相反，写法相同。

这些规则不产生名为竞争或死锁的结论。令牌仍为 0 的等待没有完成步骤；该核的后续同管道项保持关闭。另一核可以继续。

6 与 HIVM 的对应

本节的 Δ、Σ、F 和 foot 只覆盖两个例子用到的算子。复制关系 F_{copy} 按足迹中的对应单元传递当前内容，包括 Uninit。

HIVM	核心形式	实例
`arith`、视图构造、`get_block_idx`	$\mathsf{scalar}[g]$	Δ 与 eval
`load`	$\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},\ldots)$	F_{copy}，GM 到 UB
`store`	$\mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},\ldots)$	F_{copy}，UB 到 GM
`fixpipe`	$\mathsf{issue}[\mathsf{FIX}](\mathsf{Fixpipe},\ldots)$	F_{copy}，L0C 到 GM
向量函数 `vf_3`	$\mathsf{issue}[\mathsf{V}](\mathsf{MulAdd},\ldots)$	三块输入都是值时按乘加写入目的地；含 Uninit 时目的地记为 Uninit
`set_flag`、`wait_flag`、`pipe_barrier`	同名核内命令	本核的 φ_d 与队列
`sync_block_set`、`sync_block_wait`	syncblockset、syncblockwait	共用的 β

6.1 一个 AIV 上的乘加

`debug-memory.mlir` 中的 `@mul_add_kernel` 是单独的 AIV，没有 `sync_block`。三次装载分别使用事件 $0,1,2$。下面写出事件 0 的一次装载，以及随后的乘加和两次写回；另外两次装载只更换事件和视图。源程序里由运行时决定的 `scf.if` 在进入该项之前取定分支，`set_ctrl` 不进入语言。动态长度已经算进视图。

$$
\begin{aligned}
&\mathsf{aiv}\{\mathsf{setflag}[\mathsf{V},\mathsf{MTE2}](\varepsilon_0);\\
&\quad \mathsf{waitflag}[\mathsf{V},\mathsf{MTE2}](\varepsilon_0);\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},g_0,u_0);\\
&\quad \mathsf{setflag}[\mathsf{MTE2},\mathsf{V}](\varepsilon_0);\mathsf{waitflag}[\mathsf{MTE2},\mathsf{V}](\varepsilon_0);\\
&\quad \mathsf{issue}[\mathsf{V}](\mathsf{MulAdd},u_0,u_1,u_2,u_3);\\
&\quad \mathsf{setflag}[\mathsf{V},\mathsf{MTE3}](\varepsilon_0);\mathsf{waitflag}[\mathsf{V},\mathsf{MTE3}](\varepsilon_0);\\
&\quad \mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},u_3,g_3);\mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},u_3,g_4);\\
&\quad \mathsf{pipebarrier}[\mathsf{ALL}];\mathsf{return}()\}.
\end{aligned}
$$

`vf_3` 内部的 AVE 装载、乘法和加法都包在同一次 MulAdd 里。去掉 $\mathsf{MTE2}\to\mathsf{V}$ 这一对后，乘加项仍可在装载完成前打开并完成。它读到的是当时 UB 中的值。若那些单元仍是装载前的内容，写回 Global Memory 的就是这些内容的乘加结果。这是一次普通的完成步骤。

6.2 AIC 与 AIV 的一次交接

这次交接取自 `attention_hivm_lowered.mlir`。AIC 一侧的函数是 `_attn_fwd_mix_aic`。它用 `fixpipe` 写入 workspace，再执行 `sync_block_set`：核类型为 `CUBE`，管道为 `PIPE_FIX` 与 `PIPE_S`，flag 为 0。AIV 一侧的函数是 `_attn_fwd_mix_aiv`。它先执行管道和 flag 相同、核类型为 `VECTOR` 的 `sync_block_wait`，再把 workspace 装入 UB。源指令中的核类型由命令所在的一侧表达。下面只保留这一次交接，不展开 `mmadL1`、`nd2nz` 和 FFTS 基址。L0C 中的值是 `fixpipe` 的输入。

$$
\begin{aligned}
&\mathsf{aic}\{\mathsf{issue}[\mathsf{FIX}](\mathsf{Fixpipe},\ell,w);\mathsf{syncblockset}[\mathsf{FIX},\mathsf{S}](\varepsilon_0);\mathsf{return}()\}\\
&\quad\|\ \mathsf{aiv}\{\mathsf{syncblockwait}[\mathsf{FIX},\mathsf{S}](\varepsilon_0);\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},w,u);\mathsf{return}()\}.
\end{aligned}
$$

syncblockset 要等同一条 `PIPE_FIX` 队列里排在它前面的 `fixpipe` 离开之后才把 $\beta(\mathsf{FIX},\mathsf{S},\varepsilon_0)$ 置为 1。AIV 的等待管道是 S，因此在令牌变为 1 之前，AIV 不能发射后面的 `load`。去掉这条 syncblockwait 后，AIV 可以先发射并完成装载，读到 `fixpipe` 写入之前 workspace 里的内容。